

Back To Practice

< Q1 >

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n;
5
6     // Input for memory blocks
7     printf("Enter number of memory blocks:\n");
8     scanf("%d", &m);
9
10    int blockSize[m];
11    int originalBlockSize[m];
12
13    printf("Enter size of each block:\n");
14    for (int i = 0; i < m; i++) {
15        scanf("%d", &blockSize[i]);
16        originalBlockSize[i] = blockSize[i];
17    }
18
19    // Input for processes
20    printf("Enter number of processes:\n");
21    scanf("%d", &n);
22
23    int processSize[n];
24
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Completed

Select Language : C (GCC 9.2.0)

```
25 printf("Enter size of each process:\n");
26 for (int i = 0; i < n; i++) {
27     scanf("%d", &processSize[i]);
28 }
29
30 // First Fit Allocation Logic
31 for (int i = 0; i < n; i++) {
32     int allocated = 0;
33
34     for (int j = 0; j < m; j++) {
35         if (blockSize[j] >= processSize[i]) {
36
37             int fragment = blockSize[j] - processSize[i];
38
39             printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
40                 i + 1, processSize[i], j + 1, originalBlockSize[j], fragment);
41
42             blockSize[j] -= processSize[i];
43
44             allocated = 1;
45             break;
46         }
47     }
48 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Save

Back To Practice

< Q1 >

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```

32     int allocated = 0;
33
34     for (int j = 0; j < m; j++) {
35         if (blockSize[j] >= processSize[i]) {
36
37             int fragment = blockSize[j] - processSize[i];
38
39             printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
40                 i + 1, processSize[i], j + 1, originalBlockSize[j], fragment);
41
42             blockSize[j] -= processSize[i];
43
44             allocated = 1;
45             break;
46         }
47     }
48
49     if (!allocated) {
50         printf("Process %d of size %d is not allocated\n", i + 1, processSize[i]);
51     }
52 }
53
54 return 0;
55 }
```

Test Cases 2

Run Sample Cases

Run All Cases